RDF 1.2 TriG

RDF Dataset Language

W3C Working Draft 13 November 2025

▼ More details about this document

This version:
https://www.w3.org/TR/2025/WD-rdf12-trig-20251113/

Latest published version:
https://www.w3.org/TR/rdf12-trig/

Latest editor's draft:
https://w3c.github.io/rdf-trig/spec/

History:
https://www.w3.org/standards/history/rdf12-trig/
Commit history

Test suite:
https://w3c.github.io/rdf-tests/rdf/rdf12/rdf-trig/

Latest Recommendation:
https://www.w3.org/TR/trig

Editors:
Gregg Kellogg (until 2025-09-06), in memoriam
Dominik Tomaszuk

Former editors:
Gavin Carothers (RDF 1.1)
Andy Seaborne (RDF 1.1)

Authors:
Chris Bizer (Freie Universität Berlin)
Richard Cyganiak (Freie Universität Berlin)

Feedback:
GitHub w3c/rdf-trig (pull requests, new issue, open issues)
public-rdf-star-wg@w3.org with subject line `[rdf12-trig] … message topic …` (archives)

Copyright © 2010-2025 World Wide Web Consortium. W3C® liability, trademark and permissive document license rules apply.

Abstract

This document defines a textual syntax for RDF called TriG that allows an RDF dataset to be completely written in a compact and natural text form, with abbreviations for common usage patterns and datatypes. TriG is an extension of the Turtle [*RDF12-TURTLE*] format.

RDF 1.2 TriG shares triple terms with [*RDF12-TURTLE*] as a fourth kind of RDF term which can be used as the object of another triple, making it possible to make statements about other statements. RDF 1.2 TriG also adds shares directional language-tagged strings with [*RDF12-TURTLE*].

In addition, RDF 1.2 TriG shares the reifying triples and annotation syntax extensions with [*RDF12-TURTLE*] which allows triple terms to also be asserted.

Status of This Document

This section describes the status of this document at the time of its publication. A list of current W3C publications and the latest revision of this technical report can be found in the W3C standards and drafts index.

This document is part of the RDF 1.2 document suite. TriG is intended the meet the charter requirement of the RDF & SPARQL Working Group to define an RDF syntax for multiple graphs. TriG is an extension of the Turtle syntax for RDF [*RDF12-TURTLE*]. The current document is based on the original proposal by Chris Bizer and Richard Cyganiak.

This document was published by the RDF & SPARQL Working Group as a Working Draft using the Recommendation track.

Publication as a Working Draft does not imply endorsement by W3C and its Members.

This is a draft document and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to cite this document as other than a work in progress. Future updates to this upcoming Recommendation may incorporate new features.

This document was produced by a group operating under the W3C Patent Policy. W3C maintains a public list of any patent disclosures made in connection with the deliverables of the group; that page also includes instructions for disclosing a patent. An individual who has actual knowledge of a patent that the individual believes contains Essential Claim(s) must disclose the information in accordance with section 6 of the W3C Patent Policy.

This document is governed by the 18 August 2025 W3C Process Document.

§ Set of Documents

This document is one of eleven RDF 1.2 and twelve SPARQL 1.2 documents produced by the RDF & SPARQL Working Group.

▶ List of documents

Table of Contents

Abstract

Status of This Document
Set of Documents

1. Introduction

2. TriG Language
2.1 Version Announcement
2.2 Triple Statements
2.3 Graph Statements
2.4 Triple Terms
2.5 Other Terms
2.5.1 Special Considerations for Blank Nodes

3. Conformance
3.1 Media Type and Content Encoding

4. TriG Grammar
4.1 White Space
4.2 Comments
4.3 IRI References
4.4 Escape Sequences
4.5 Grammar
4.6 Selected Terminal Literal Strings

5. Parsing
5.1 Parser State

5.2 RDF Term Constructors
5.3 RDF Triples Construction
5.3.1 Output Graph
5.3.2 Triple Output
5.3.3 Triple Production
5.3.4 Reifiers
5.3.5 Reified Triples
5.3.6 Annotations
5.3.7 Annotation Blocks
5.3.8 Property Lists
5.3.9 Collections

A. Privacy Considerations

B. Security Considerations

C. Internet Media Type, File Extension and Macintosh File Type

D. Acknowledgments
D.1 Acknowledgments for RDF 1.1
D.2 Acknowledgments for RDF 1.2

E. Changes between RDF 1.1 and RDF 1.2

F. Index
F.1 Terms defined by this specification
F.2 Terms defined by reference

G. References
G.1 Normative references
G.2 Informative references

§ 1. Introduction

This section is non-normative.

This document defines TriG, a concrete syntax for RDF as defined in the RDF Concepts and Abstract Syntax document [*RDF12-CONCEPTS*]. TriG is an extension of Turtle [*RDF12-TURTLE*], extended to

support representing a complete RDF dataset.

§ 2. TriG Language

This section is non-normative.

A TriG document allows writing down an RDF dataset in a compact textual form. It consists of a sequence of directives, triple statements, graph statements which contain triple-generating statements and optional blank lines. Comments may be given after a # that is not part of another lexical token and continue to the end of the line.

Graph statements are a pair of an IRI or blank node and a group of triple statements surrounded by braces ({}). The IRI or blank node of the graph statement may be used in another graph statement which implies taking the union of the tripes generated by each graph statement. An IRI or blank node used as a graph label may also reoccur as part of any triple statement. Optionally a graph statement may not not be labeled with an IRI. Such a graph statement corresponds to the Default Graph of an RDF dataset.

The construction of an RDF dataset from a TriG document is defined in 4. TriG Grammar and 5. Parsing.

§ 2.1 Version Announcement

TriG uses the same version announcement mechanism as Turtle [*RDF12-TURTLE*].

§ 2.2 Triple Statements

As TriG is an extention of the Turtle language it allows for any constructs from the Turtle language. Simple Triples, Predicate Lists, and Object Lists can all be used either inside a graph statement, or on their own as in a TriG document. When outside a graph statement, the triples are considered to be part of the default graph of the RDF dataset.

§ 2.3 Graph Statements

A graph statement pairs an IRIs or blank node with an RDF graph. The triple statements that make up the graph are enclosed in {}.

In a TriG document a graph IRI or blank node may be used as label for more than one graph statements. The graph label of a graph statement may be omitted. In this case the graph is considered the default graph of the RDF dataset.

A RDF dataset might contain only a single graph.

EXAMPLE 1: Dataset with single graph

```
# This document encodes one graph.
PREFIX ex: <http://www.example.org/vocabulary#>
PREFIX : <http://www.example.org/exampleDocument#>

:G1 { :Monica a ex:Person ;
    ex:name "Monica Murphy" ;
    ex:homepage <http://www.monicamurphy.org> ;
    ex:email <mailto:monica@monicamurphy.org> ;
    ex:hasSkill ex:Management ,
                ex:Programming .
}
```

An RDF dataset may contain a default graph and/or zero or more named graphs.

EXAMPLE 2: Dataset with default and named graphs

```
# This document contains a default graph and two named graphs.

PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX dc: <http://purl.org/dc/terms/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

# default graph
{
  <http://example.org/bob> dc:publisher "Bob" .
  <http://example.org/alice> dc:publisher "Alice" .
}

<http://example.org/bob> {
   _:a foaf:name "Bob" .
   _:a foaf:mbox <mailto:bob@oldcorp.example.org> .
   _:a foaf:knows _:b .
}

<http://example.org/alice> {
   _:b foaf:name "Alice" .
   _:b foaf:mbox <mailto:alice@work.example.org> .
}
```

TriG provides various alternative ways to write graphs and triples, giving the data writer choices for clarity:

EXAMPLE 3: Alternative ways to write named graphs

```
# This document contains a same data as the previous example.

PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX dc: <http://purl.org/dc/terms/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

# default graph - no {} used.
<http://example.org/bob> dc:publisher "Bob" .
<http://example.org/alice> dc:publisher "Alice" .

# GRAPH keyword to highlight a named graph
# Abbreviation of triples using ;
GRAPH <http://example.org/bob>
{
   [] foaf:name "Bob" ;
      foaf:mbox <mailto:bob@oldcorp.example.org> ;
      foaf:knows _:b .
}

GRAPH <http://example.org/alice>
{
    _:b foaf:name "Alice" ;
        foaf:mbox <mailto:alice@work.example.org>
}
```

§ 2.4 Triple Terms

TriG shares the same syntax for representing triple terms as Turtle, including Reifying Triples and the Annotation Syntax.

EXAMPLE 4: Triple Term

```
VERSION "1.2"
PREFIX :    <http://www.example.org/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

                        _:e38  :familyName  "Smith" .
_:anno rdf:reifies <<( _:e38  :jobTitle    "Designer" )>> .
                        _:anno :accordingTo _:e22 .
```

2.5 Other Terms

All other terms and directives come from Turtle.

2.5.1 Special Considerations for Blank Nodes

Blank nodes sharing the same identifier in differently labeled graph statements are considered to be the same blank node.

3. Conformance

As well as sections marked as non-normative, all authoring guidelines, diagrams, examples, and notes in this specification are non-normative. Everything else in this specification is normative.

The key words *MUST*, *MUST NOT*, and *SHOULD* in this document are to be interpreted as described in BCP 14 [*RFC2119*] [*RFC8174*] when, and only when, they appear in all capitals, as shown here.

This specification defines conformance criteria for:

- TriG documents
- TriG parsers

A conforming ***TriG document*** is a Unicode string that conforms to the grammar and additional constraints defined in 4. TriG Grammar, starting with the `trigDoc` production. A TriG document serializes an RDF dataset.

A conforming ***TriG parser*** is a system capable of reading TriG documents on behalf of an application. It makes the serialized RDF dataset, as defined in 5. Parsing, available to the application, usually through some form of API.

The IRI that identifies the TriG language is: `http://www.w3.org/ns/formats/TriG`

> NOTE
>
> This specification does not define how TriG parsers handle non-conforming input documents.

§ 3.1 Media Type and Content Encoding

The media type of TriG is `application/trig`. The content encoding of TriG content is always UTF-8.

§ 4. TriG Grammar

A TriG document is an RDF string encoded in UTF-8 [*RFC3629*]. Only Unicode scalar values, in the ranges `U+0000` to `U+D7FF` and `U+E000` to `U+10FFFF`, are allowed. This excludes surrogate code points, range `U+D800` to `U+DFFF`.

§ 4.1 White Space

White space (production `WS`) is used to separate two terminals which would otherwise be (mis-)recognized as one terminal. Rule names below in capitals indicate where white space is significant; these form a possible choice of terminals for constructing a TriG parser.

White space is significant in the production `String`.

§ 4.2 Comments

Comments in TriG start with a `#` outside an `IRIREF`, `STRING_LITERAL_SINGLE_QUOTE`, `STRING_LITERAL_QUOTE`, `STRING_LITERAL_LONG_SINGLE_QUOTE`, or `STRING_LITERAL_LONG_QUOTE`, and continue to the end of line (marked by `LFs`, or `CRs`), or end of file if there is no end of line after the comment marker. Comments are treated as white space.

§ 4.3 IRI References

Relative IRI references are resolved with base IRIs as per *Uniform Resource Identifier (URI): Generic Syntax* [*RFC3986*] using only the basic algorithm in section 5.2. Neither Syntax-Based Normalization nor Scheme-Based Normalization (described in sections 6.2.2 and 6.2.3 of RFC3986) are performed. Characters additionally allowed in IRI references are treated in the same way that unreserved characters are treated in URI references, per section 6.5 of *Internationalized Resource Identifiers (IRIs)* [*RFC3987*].

The `@base` or `BASE` directive defines the Base IRI used to resolve relative IRI references per [*RFC3986*] section 5.1.1, "Base URI Embedded in Content". Section 5.1.2, "Base URI from the Encapsulating Entity" defines how the In-Scope Base IRI may come from an encapsulating document, such as a SOAP envelope with an `xml:base` directive or a MIME multipart document with a `Content-Location` header. The "Retrieval URI" identified in 5.1.3, Base "URI from the Retrieval URI", is the URL from which a particular TriG document was retrieved. If none of the above specifies the Base URI, the default Base URI (section 5.1.4, "Default Base URI") is used. Each `@base` or `BASE` directive sets a new In-Scope Base URI, relative to the previous one.

§ 4.4 Escape Sequences

There are three forms of escapes used in TriG documents:

- A ***numeric escape sequence*** represents the value of a Unicode code point.

  A numeric escape sequence *MUST NOT* produce a code point value in the range `U+D800` to `U+DFFF`, which is the range for Unicode surrogates.

Escape sequence	Unicode code point
`\u` hex hex hex hex	A Unicode code point in the ranges `U+0000` to `U+D7FF` and `U+E000` to `U+D7FF`, corresponding to the value encoded by the four hexadecimal digits interpreted from most significant to least significant digit.
`\U` hex hex hex hex hex hex hex hex	A Unicode code point in the ranges `U+0000` to `U+D7FF` and `U+E000` to `U+10FFFF`, corresponding to the value encoded by the eight hexadecimal digits interpreted from most significant to least significant digit.

where hex is a hexadecimal character

```
HEX ::= [0-9] | [A-F] | [a-f]
```

- A ***string escape sequence*** represents a character traditionally escaped in string literals:

Escape sequence	Unicode code point
`\t`	`U+0009`
`\b`	`U+0008`
`\n`	`U+000A`
`\r`	`U+000D`
`\f`	`U+000C`
`\"`	`U+0022`
`\'`	`U+0027`
`\\`	`U+005C`

- A ***reserved character escape sequence*** consists of a `\` followed by one of these characters `~.-!$&'()*+,;=/?#@%_`, and represents the character to the right of the `\`.

Context where each kind of escape sequence can be used

	numeric escapes	**string escapes**	**reserved character escapes**
IRIs, used as RDF terms or as in `PREFIX` or `BASE` declarations	yes	no	no
local names	no	no	yes
Strings	yes	yes	no

NOTE

%-encoded sequences are in the character range for IRIs and are explicitly allowed in local names. These appear as a '%' followed by two hex characters and represent that same sequence of three characters. These sequences are *not* decoded during processing. A term written as `<http://a.example/%66oo-bar>` in TriG designates the IRI `http://a.example/%66oo-bar` and not IRI `http://a.example/foo-bar`. A term written as `ex:%66oo-bar` with a prefix `PREFIX ex: <http://a.example/>` also designates the IRI `http://a.example/%66oo-bar`.

§ 4.5 Grammar

The EBNF used here is defined in XML 1.0 [*EBNF-NOTATION*].

Notes:

1. A blank node identifier represents the same blank node throughout the TriG document.
2. Keywords in single quotes ('`@base`', '`@prefix`', '`@version`', '`a`', '`true`', and '`false`') are case-sensitive. Keywords in quotation marks ("`BASE`", "`PREFIX`"), and "`VERSION`") are case-insensitive.
3. Escape sequences `UCHAR` and `ECHAR` are case sensitive.
4. When tokenizing the input and choosing grammar rules, the longest match is chosen.
5. The TriG grammar is LL(1) and LALR(1) when the rules with uppercased names are used as terminals.
6. The entry point into the grammar is `trigDoc`.
7. In signed numbers, no white space is allowed between the sign and the number.
8. The strings '`@prefix`', '`@base`', and '`@version`' match the pattern for `LANG_DIR`, though neither `prefix`, `base`, nor `version` are registered language subtags. This specification does not define whether a quoted literal followed by either of these tokens (e.g., `"A"@base`) is in the Turtle language.

```
[1]  trigDoc              ::= (directive | block)*
[2]  block                ::= triplesOrGraph | wrappedGraph |
                              triples2 | ("GRAPH" labelOrSubject
                              wrappedGraph)
[3]  triplesOrGraph       ::= (labelOrSubject (wrappedGraph |
                              (predicateObjectList '.'))) |
                              (reifiedTriple predicateObjectList? '.')
```

```
[4]  triples2                ::= (blankNodePropertyList predicateObjectList? '.') | (collection predicateObjectList '.')
[5]  wrappedGraph            ::= '{' triplesBlock? '}'
[6]  triplesBlock            ::= triples ('.' triplesBlock?)?
[7]  labelOrSubject          ::= iri | BlankNode
[8]  directive               ::= prefixID | base | version | sparqlPrefix | sparqlBase | sparqlVersion
[9]  prefixID                ::= '@prefix' PNAME_NS IRIREF '.'
[10] base                    ::= '@base' IRIREF '.'
[11] version                 ::= '@version' VersionSpecifier '.'
[12] sparqlPrefix            ::= "PREFIX" PNAME_NS IRIREF
[13] sparqlBase              ::= "BASE" IRIREF
[14] sparqlVersion           ::= "VERSION" VersionSpecifier
[15] VersionSpecifier        ::= STRING_LITERAL_QUOTE | STRING_LITERAL_SINGLE_QUOTE
[16] triples                 ::= (subject predicateObjectList) | (blankNodePropertyList predicateObjectList?) | (reifiedTriple predicateObjectList?)
[17] predicateObjectList     ::= verb objectList (';' (verb objectList)?)*
[18] objectList              ::= object annotation (',' object annotation)*
[19] verb                    ::= predicate | 'a'
[20] subject                 ::= iri | BlankNode | collection
[21] predicate               ::= iri
[22] object                  ::= iri | BlankNode | collection | blankNodePropertyList | literal | tripleTerm | reifiedTriple
[23] literal                 ::= RDFLiteral | NumericLiteral | BooleanLiteral
[24] blankNodePropertyList   ::= '[' predicateObjectList ']'
[25] collection              ::= '(' object* ')'
[26] NumericLiteral          ::= INTEGER | DECIMAL | DOUBLE
[27] RDFLiteral              ::= String (LANG_DIR | ('^^' iri))?
[28] BooleanLiteral          ::= 'true' | 'false'
[29] String                  ::= STRING_LITERAL_QUOTE | STRING_LITERAL_SINGLE_QUOTE |
```

STRING_LITERAL_LONG_SINGLE_QUOTE | STRING_LITERAL_LONG_QUOTE

[30] iri ::= IRIREF | PrefixedName

[31] PrefixedName ::= PNAME_LN | PNAME_NS

[32] BlankNode ::= BLANK_NODE_LABEL | ANON

[33] reifier ::= '~' (iri | BlankNode)?

[34] reifiedTriple ::= '<<' rtSubject verb rtObject reifier? '>>'

[35] rtSubject ::= iri | BlankNode | reifiedTriple

[36] rtObject ::= iri | BlankNode | literal | tripleTerm | reifiedTriple

[37] tripleTerm ::= '<<(' ttSubject verb ttObject ')>>'

[38] ttSubject ::= iri | BlankNode

[39] ttObject ::= iri | BlankNode | literal | tripleTerm

[40] annotation ::= (reifier | annotationBlock)*

[41] annotationBlock ::= '{|' predicateObjectList '|}'

§ Productions for terminals

[43] IRIREF ::= '<' ([^#x00-#x20<>"{}|^`\] | UCHAR)* '>' /* #x00=NULL #x01-#x1F=control codes #x20=space */

[44] PNAME_NS ::= PN_PREFIX? ':'

[45] PNAME_LN ::= PNAME_NS PN_LOCAL

[46] BLANK_NODE_LABEL ::= '_:' (PN_CHARS_U | [0-9]) ((PN_CHARS | '.')* PN_CHARS)?

[47] LANG_DIR ::= '@' [a-zA-Z]+ ('-' [a-zA-Z0-9]+)* ('--' [a-zA-Z]+)?

[48] INTEGER ::= [+-]? [0-9]+

[49] DECIMAL ::= [+-]? ([0-9]* '.' [0-9]+)

[50] DOUBLE ::= [+-]? (([0-9]+ '.' [0-9]* EXPONENT) | ('.' [0-9]+ EXPONENT) | ([0-9]+ EXPONENT))

[51] EXPONENT ::= [eE] [+-]? [0-9]+

[52] STRING_LITERAL_QUOTE ::= '"' ([^#x22#x5C#x0A#x0D] | ECHAR | UCHAR)* '"'

[53] STRING_LITERAL_SINGLE_QUOTE ::= "'" ([^#x27#x5C#x0A#x0D] | ECHAR | UCHAR)* "'"

[54] STRING_LITERAL_LONG_SINGLE_QUOTE ::= "'''" (("'" | "''")? ([^'\] | ECHAR | UCHAR))* "'''"

[55] STRING_LITERAL_LONG_QUOTE ::= '"""' (('"' | '""')? ([^"\] | ECHAR |

```
                                    UCHAR)* '"""'
[56] UCHAR                      ::= ('\u' HEX HEX HEX HEX) | ('\U' HEX HEX
                                    HEX HEX HEX HEX HEX HEX)
[57] ECHAR                      ::= '\' [tbnrf\"']
[58] WS                         ::= #x20 | #x09 | #x0D | #x0A
[59] ANON                       ::= '[' WS* ']'
[60] PN_CHARS_BASE              ::= [A-Z]
                                |   [a-z]
                                |   [#xC0-#xD6]
                                |   [#xD8-#xF6]
                                |   [#xF8-#x02FF]
                                |   [#x0370-#x037D]
                                |   [#x037F-#x1FFF]
                                |   [#x200C-#x200D]
                                |   [#x2070-#x218F]
                                |   [#x2C00-#x2FEF]
                                |   [#x3001-#xD7FF]
                                |   [#xF900-#xFDCF]
                                |   [#xFDF0-#xFFFD]
                                |   [#x00010000-#x000EFFFF]
[61] PN_CHARS_U                 ::= PN_CHARS_BASE | '_'
[62] PN_CHARS                   ::= PN_CHARS_U | '-' | [0-9] | #xB7 |
                                    [#x0300-#x036F] | [#x203F-#x2040]
[63] PN_PREFIX                  ::= PN_CHARS_BASE ((PN_CHARS | '.')*
                                    PN_CHARS)?
[64] PN_LOCAL                   ::= (PN_CHARS_U | ':' | [0-9] | PLX)
                                    ((PN_CHARS | '.' | ':' | PLX)* (PN_CHARS
                                    | ':' | PLX))?
[65] PLX                        ::= PERCENT | PN_LOCAL_ESC
[66] PERCENT                    ::= '%' HEX HEX
[67] HEX                        ::= [0-9] | [A-F] | [a-f]
[68] PN_LOCAL_ESC               ::= '\' ('_' | '~' | '.' | '-' | "!" | '$' |
                                    '&' | "'" | '(' | ')' | '*' | '+' | ',' |
                                    ';' | '=' | '/' | '?' | '#' | '@' | '%')
```

A text version of this grammar is available here.

§ 4.6 Selected Terminal Literal Strings

This document uses some specific terminal literal strings [*EBNF-NOTATION*]. To clarify the Unicode code points used for these terminal literal strings, the following table describes specific characters and sequences used throughout this document.

Code	Glyph	Description
U+000A	LF	Line feed
U+000D	CR	Carriage return
U+0022	"	Quotation mark
U+0023	#	Number sign
U+0027	'	Apostrophe
U+002D	-	Hyphen
U+003B	:	Colon
U+0040	@	At sign
U+005C	\	Backslash
U+005F	_	Underscore
U+0061	a	Latin small letter E
U+007B	{	Left curly bracket
U+007D	}	Right curly bracket

Other short terminal literal strings are composed of specific sequences of Unicode characters:

space
: U+0020

"""
: three concatenated quotation mark characters, each having the code point U+0022

'''
: three concatenated apostrophe characters, each having the code point U+0027

--
: two concatenated - characters

§ 5. Parsing

The RDF Concepts and Abstract Syntax [*RDF12-CONCEPTS*] specification defines four types of RDF term: IRIs, literals, blank nodes, and triple terms. Literals are composed of a lexical form and an optional language tag [*BCP47*] – possibly including an initial text direction – or an optional datatype IRI. An extra type, `prefix`, is used during parsing to map string identifiers to namespace IRIs. This section maps a string conforming to the grammar in 4.5 Grammar to a set of triples by mapping strings matching productions and lexical tokens to RDF terms or their components (e.g., language tags, lexical forms of literals). Grammar productions change the parser state and emit triples.

§ 5.1 Parser State

Parsing TriG requires a state of ten items:

- IRI *baseURI* — When the `base` production is reached, the second rule argument, `IRIREF`, is the base URI used for relative IRI reference resolution.
- Map[prefix -> IRI] *namespaces* — The second and third rule arguments (`PNAME_NS` and `IRIREF`) in the prefixID production assign a namespace name (`IRIREF`) for the prefix (`PNAME_NS`). Outside of a `prefixID` production, any `PNAME_NS` is substituted with the namespace. Note that the prefix may be an empty string, per the `PNAME_NS` production: `PN_PREFIX? ":"`.
- Map[string -> blank node] *bnodeLabels* — A mapping from a string to a blank node.
- RDF Term *curSubject* — The *curSubject* is bound to the `subject`, `rtSubject`, `ttSubject`, `blankNodePropertyList`, `collection`, `tripleTerm`, and `annotationBlock` productions.
- RDF_Term *curPredicate* — The *curPredicate* is bound to the the `verb` production. If token matched was `a`, *curPredicate* is bound to the IRI `http://www.w3.org/1999/02/22-rdf-syntax-ns#type`.
- RDF Term *curObject* — The *curObject* is bound to the `object`, `rtObject`, and `ttObject` productions.
- RDF Term *curReifier* — The *curReifier* is bound to the `reifier` and `annotationBlock` productions.
- Triple term *curTripleTerm* — The *curTripleTerm* is set in the Annotations constructor.
- RDF_Term *curGraph* — The *curGraph* is bound to the label of the graph that is the destination of triples produced in parsing. When undefined, triples are destined for the default graph.

- `xsd:string` *curVersion* – The RDF version used for parsing the document into Triples. If specified as part of a Media Type, the default value for *curVersion* is taken from the `version` parameter. The only defined value is `"1.2"`. Parser behavior for other version values is undefined. This specification does not define behavior if different versions are provided for *curVersion*.

Term Constructors can create a stack of these values indicated by using language such as "records the *curSubject* and *curPredicate*."

§ 5.2 RDF Term Constructors

This table maps productions and lexical tokens to RDF terms or components of RDF terms listed in 5. Parsing:

production	type	procedure
IRIREF	IRI	The characters between < and > are taken, with the numeric escape sequences unescaped, to form the IRI. Relative IRI reference resolution is performed per Section 6.3. The resulting IRI *MUST* comply with the syntactic restrictions of generic IRI syntax, and *SHOULD* conform to section 3.3 of [*RFC3986*] and comply with any narrower restrictions imposed by the corresponding IRI scheme specification.
	prefix	When used in a `prefixID` or `sparqlPrefix` production, the `prefix` is the potentially empty RDF string that matches the first argument of the rule. That

PNAME_NS		string serves as a key in the namespaces map; the expanded second argument is stored under that key for future lookup.
	IRI	When used in a `PrefixedName` production, the `iri` is the value in the namespaces map corresponding to the first argument of the rule.
PNAME_LN	IRI	A potentially empty prefix is identified by the first sequence, `PNAME_NS`. The namespaces map *MUST* have a corresponding `namespace`. The RDF string of the IRI is formed by unescaping the reserved characters in the second argument, `PN_LOCAL`, and concatenating this onto the `namespace`. The resulting IRI *MUST* comply with the syntactic restrictions of generic IRI syntax, and *SHOULD* comply with any narrower restrictions imposed by the corresponding IRI scheme specification.
VersionSpecifier	literal	The *curVersion* is taken from a literal using the matched RDF string lexical form and `xsd:string` datatype.
		The characters between the

STRING_LITERAL_SINGLE_QUOTE	lexical form	outermost '_s are taken, with numeric and string escape sequences unescaped, to form the Unicode string of a lexical form.
STRING_LITERAL_QUOTE	lexical form	The characters between the outermost "_s are taken, with numeric and string escape sequences unescaped, to form the Unicode string of a lexical form.
STRING_LITERAL_LONG_SINGLE_QUOTE	lexical form	The characters between the outermost '''_s are taken, with numeric and string escape sequences unescaped, to form the unicode string of a lexical form.
STRING_LITERAL_LONG_QUOTE.	lexical form	The characters between the outermost """_s are taken, with numeric and string escape sequences unescaped, to form the Unicode string of a lexical form.
LANG_DIR	language tag	The characters following the @ form the language tag and optionally the initial text direction, if the matched characters include --. The language tag *MUST* be well-formed according to section 2.2.9 of [*BCP47*]. If present, the initial text direction *MUST* be either `ltr` or `rtl`.

RDFLiteral	literal	The literal has a lexical form of the first rule argument, `String`. If the `'^^' iri` rule is matched, the datatype IRI is derived from the `iri`, and the literal has no language tag. If the `LANG_DIR` rule is matched, the language tag and initial text direction are taken from `LANG_DIR`. If there is no initial text direction, the datatype is `rdf:langString`. If there is an initial text direction, the datatype is `rdf:dirLangString`. If neither matched, the datatype is `xsd:string`, and the literal has no language tag.
INTEGER	literal	The literal has a lexical form of the input string, and a datatype of `xsd:integer`.
DECIMAL	literal	The literal has a lexical form of the input string, and a datatype of `xsd:decimal`.
DOUBLE	literal	The literal has a lexical form of the input string, and a datatype of `xsd:double`.
BooleanLiteral	literal	The literal has a lexical form of the `true` or `false`, depending on which matched the input, and a datatype of `xsd:boolean`.

BLANK_NODE_LABEL	blank node	The string matching the second argument, PN_LOCAL, is a key in bnodeLabels. If there is no corresponding blank node in the map, one is allocated.
ANON	blank node	A blank node is generated.
blankNodePropertyList	blank node	A blank node is generated. Note the rules for blankNodePropertyList in the next section.
collection	blank node	For non-empty lists, a blank node is generated. Note the rules for collection in the next section.
	IRI	For empty lists, the resulting IRI is rdf:nil. Note the rules for collection in the next section.
reifier	IRI | blank node	The *curReifier* is taken from term, which is taken from the matched iri production or BlankNode production, if any. If no such production is matched, term is taken from a fresh RDF blank node.
tripleTerm	triple term	The triple term is composed of the terms constructed from the ttSubject, predicate, and ttObject productions.

reifiedTriple	IRI | blank node	The term is taken from the matched `reifier`, if any, or from a fresh RDF blank node.
annotationBlock	IRI | blank node	The term is taken from a previously matched reifier, if any, or from a fresh RDF blank node.

> NOTE
>
> As processors which detect errors on input can result in graphs which contain fewer triples than are described in the input (including no triples whatsoever), consumers should consider information of any errors signaled when using the output triples, which may be incomplete and/or include ill-typed or ill-formed terms.

§ 5.3 RDF Triples Construction

A TriG document defines an RDF Dataset composed of one default graph and zero or more named graphs. Each graph is composed of a set of RDF triples.

§ 5.3.1 Output Graph

The state *curGraph* is initially unset. It records the label of the graph for triples produced during parsing. If undefined, the default graph is used.

The rule `labelOrSubject` sets both *curGraph* and *curSubject* (only one of these will be used).

The following grammar production clauses set *curGraph* to be undefined, indicating the default graph:

- The grammar production clause `wrappedGraph` in rule `block`
- The grammar production in rule `triples2`.

The grammar production `labelOrSubject predicateObjectList '.'` unsets *curGraph* before handling the `predicateObjectList` production in rule `triplesOrGraph`.

§ 5.3.2 Triple Output

Each RDF triple produced is added to *curGraph*, or the default graph if *curGraph* is not set at that point in the parsing process.

The `subject` production sets the *curSubject*. The `verb` production sets the *curPredicate*.

Triples are produced at the following points in the parsing process and each RDF triple produced is added to the graph identified by *curGraph*.

§ 5.3.3 Triple Production

Each object *N* in the document produces an RDF triple: *curSubject curPredicate N*.

§ 5.3.4 Reifiers

Beginning the `reifier` production, the *curReifier* is taken from the `reifier` term constructor. Then yield the the RDF triple *curReifier* `rdf:reifies` *curTripleTerm*.

§ 5.3.5 Reified Triples

Beginning the `reifiedTriple` production records the *curTripleTerm*. A new `tripleTerm` instance *curTripleTerm* is created using the `rtSubject`, `verb`, and `rtObject` productions. Finishing the `reifiedTriple` production, if the *curReifier* is not set, it is assigned a fresh RDF blank node; it next yields the RDF triple *curReifier* `rdf:reifies` *curTripleTerm*, and then restores the recorded value of the *curTripleTerm*. The node produced by matching `reifiedTriple` is the the *curReifier*.

§ 5.3.6 Annotations

Beginning the `annotation` production records the *curSubject* and *curPredicate*. A new `tripleTerm` instance *curTripleTerm* is created using the *curSubject curPredicate curObject*, and the value of the

curReifier is cleared. Finishing the `annotation` production restores the recorded values of the *curSubject* and *curPredicate*.

§ 5.3.7 Annotation Blocks

Beginning the `annotationBlock` production records the *curTripleTerm*. If the *curReifier* is not set, then it is assigned a fresh RDF blank node and the production yields the RDF triple *curReifier* `rdf:reifies` *curTripleTerm*. The *curSubject* is taken from the *curReifier* Finishing the `annotationBlock` production clears the value of the *curReifier* and restores the *curTripleTerm*.

> NOTE
>
> If the *curReifier* was already set, the reifying triple *curReifier* `rdf:reifies` *curTripleTerm* was emitted in 5.3.4 Reifiers.

§ 5.3.8 Property Lists

Beginning the `blankNodePropertyList` production records the *curSubject* and *curPredicate*, and sets *curSubject* to a novel blank node *B*. Finishing the `blankNodePropertyList` production restores *curSubject* and *curPredicate*. The node produced by matching `blankNodePropertyList` is the blank node *B*.

§ 5.3.9 Collections

Beginning the `collection` production records the *curSubject* and *curPredicate*. Each `object` in the `collection` production has a *curSubject* set to a novel blank node *B* and a *curPredicate* set to `rdf:first`. For each object $object_n$ after the first produces a triple:$object_{n-1}$ `rdf:rest` $object_n$. Finishing the `collection` production creates an additional triple `curSubject rdf:rest rdf:nil` . and restores *curSubject* and *curPredicate* The node produced by matching `collection` is the first blank node *B* for non-empty lists and `rdf:nil` for empty lists.

§ A. Privacy Considerations

This section is non-normative.

The TriG format is used to express arbitrary application data, which may include the expression of personally identifiable information (PII) or other information which could be considered sensitive. Authors publishing such information are advised to carefully consider the needs and use of publishing such information, as well as the applicable regulations for the regions where the data is expected to be consumed and potentially revealed (e.g., GDPR, CCPA, others), particularly whether authorization measures are needed for access to the data.

§ B. Security Considerations

This section is non-normative.

The `STRING_LITERAL_SINGLE_QUOTE`, `STRING_LITERAL_QUOTE`, `STRING_LITERAL_LONG_SINGLE_QUOTE`, and `STRING_LITERAL_LONG_QUOTE`, productions allows the use of unescaped control characters. Although this specification does not directly expose this content to an end user, it might be presented through a user agent, which may cause the presented text to be obfuscated due to presentation of such characters.

TriG is a general-purpose assertion language; applications may evaluate given data to infer more assertions or to dereference IRIs, invoking the security considerations of the scheme for that IRI. Note in particular, the privacy issues in [*RFC3023*] section 10 for HTTP IRIs. Data obtained from an inaccurate or malicious data source may lead to inaccurate or misleading conclusions, as well as the dereferencing of unintended IRIs. Care must be taken to align the trust in consulted resources with the sensitivity of the intended use of the data; inferences of potential medical treatments would likely require different trust than inferences for trip planning.

The TriG language is used to express arbitrary application data; security considerations will vary by domain of use. Security tools and protocols applicable to text (for example, PGP encryption, checksum validation, password-protected compression) may also be used on TriG documents. Security/privacy protocols must be imposed which reflect the sensitivity of the embedded information.

TriG can express data which is presented to the user, such as RDF Schema labels. Applications rendering strings retrieved from untrusted TriG documents, or using unescaped characters, *SHOULD* use warnings and other appropriate means to limit the possibility that malignant strings might be used to mislead the reader. The security considerations in the media type registration for XML ([*RFC3023*] section 10) provide additional guidance around the expression of arbitrary data and markup.

TriG uses IRIs as term identifiers. Applications interpreting data expressed in TriG *SHOULD* address the security issues of *Internationalized Resource Identifiers (IRIs)* [*RFC3987*] Section 8, as well as *Uniform Resource Identifier (URI): Generic Syntax* [*RFC3986*] Section 7.

Multiple IRIs may have the same appearance. Characters in different scripts may look similar (for instance, a Cyrillic "o" (code point U+043E) may appear similar to a Latin "o" (code point U+006F)). A character followed by combining characters may have the same visual representation as another character (for example, LATIN SMALL LETTER "E" (code point U+0065) followed by COMBINING ACUTE ACCENT (code point U+0301) has the same visual representation as LATIN SMALL LETTER "E" WITH ACUTE (U+00E9)). Any person or application that is writing or interpreting data in TriG must take care to use the IRI that matches the intended semantics, and avoid IRIs that may look similar. Further information about matching visually similar characters can be found in *Unicode Security Considerations* [*UNICODE-SECURITY*] and *Internationalized Resource Identifiers (IRIs)* [*RFC3987*] Section 8.

§ C. Internet Media Type, File Extension and Macintosh File Type

The Internet Media Type (formerly known as MIME Type) for TriG is "application/trig".

It is recommended that TriG files have the extension ".trig" (all lowercase) on all platforms.

The information that follows has been submitted to the Internet Engineering Steering Group (IESG) for review, approval, and registration with IANA.

Type name:
application

Subtype name:
trig

Required parameters:
None

Optional parameters:
`version` — this parameter is required when using RDF 1.2-specific features. If present, the value of `version` is always `"1.2"`.

Encoding considerations:
The syntax of TriG is expressed over code points in Unicode [*UNICODE*]. The encoding is always UTF-8 [*UTF-8*].

Unicode code points may also be expressed using an \uXXXX (U+0000 to U+FFFF) or \UXXXXXXXX syntax (for U+10000 onwards) where X is a hexadecimal digit [0-9A-Fa-f]

Security considerations:
See B. Security Considerations.

Interoperability considerations:

There are no known interoperability issues.

Published specification:

This specification.

Applications which use this media type:

TriG is used widely for representing RDF data. There are implementations available in most common programming languages.

Additional information:

Magic number(s):

TriG documents may have the strings 'prefix' or 'base' (case independent) near the beginning of the document.

File extension(s):

.trig

Macintosh file type code(s):

TEXT

Base URI:

The TriG base directive can change the current base URI for relative IRI references in the language that are used sequentially later in the document.

Person & email address to contact for further information:

W3C RDF & SPARQL Working Group <public-rdf-star-wg@w3.org>

Intended usage:

Common

Restrictions on usage:

None

Author(s):

The TriG specification is the product of the RDF & SPARQL WG. The W3C reserves change control over this specifications.

§ D. Acknowledgments

This section is non-normative.

§ D.1 Acknowledgments for RDF 1.1

This section is non-normative.

The editors gratefully acknowledge the work of Chris Bizer and Richard Cyganiak in creating the original TriG specification. Valuable contributions to this version were made by Gregg Kellogg, Eric Prud'hommeaux and Sandro Hawke.

The document was improved through the review process by the wider community.

D.2 Acknowledgments for RDF 1.2

This section is non-normative.

In addition to the editors, the following people have contributed to this specification: Denis Ah-Kang, Peter F. Patel-Schneider, Pierre-Antoine Champin, and Ted Thibodeau Jr

Members of the RDF & SPARQL Working Group Group included Vladimir Alexiev, James Anderson, Amin Anjomshoaa, Julián Arenas-Guerrero, Dörthe Arndt, Bilal Ben Mahria, Erich Bremer, Dan Brickley, Kurt Cagle, Sarven Capadisli, Rémi Ceres, Pierre-Antoine Champin, David Chaves-Fraga, Souripriya Das, Daniil Dobriy, Enrico Franconi, Jeffrey Phillips Freeman, Fabien Gandon, Benjamin Goering, Damien Graux, Adrian Gschwend, Olaf Hartig, Timothée Haudebourg, Ian Horrocks, Gregg Kellogg, Mark Kim, Jose Emilio Labra Gayo, Ora Lassila, Richard Lea, Niklas Lindström, Pasquale Lisena, Thomas Lörtsch, Matthew Nguyen, Peter Patel-Schneider, Thomas Pellissier Tanon, Dave Raggett, Jean-Yves ROSSI, Felix Sasaki, Andy Seaborne, Alan Snyder, Stuart Sutton, Ruben Taelman, Ted Thibodeau Jr, Dominik Tomaszuk, Raphaël Troncy, William Van Woensel, Gregory Williams, Jesse Wright, Achille Zappa, and Antoine Zimmermann.

> EDITOR'S NOTE
>
> Recognize members of the Task Force? Not an easy to find list of contributors.

E. Changes between RDF 1.1 and RDF 1.2

This section is non-normative.

This section describes the main differences from the RDF 1.1 Recommendation.

- Syntax is aligned to the Turtle [*RDF12-TURTLE*] recommendation for RDF terms.

- Separated B. Security Considerations from C. Internet Media Type, File Extension and Macintosh File Type and updated language.
- Changes the `LANGTAG` terminal production to `LANG_DIR` to include an optional initial text direction.
- Added Triple Terms for representing triple terms in TriG.
- Removed the `NIL` terminal production from the grammar, which was unused.
- Added 2.1 Version Announcement and parser state and productions to announce the RDF version associated with the input document. Note that this also includes a provision that `@version` does not match `LANG_DIR`; the previous version of the grammar would allow such a match, but would result in a literal with an ill-formed language tag according to [*BCP47*], and form an invalid literal.

§ F. Index

§ F.1 Terms defined by this specification

numeric escape sequence §4.4

reserved character escape sequence §4.4

string escape sequence §4.4

TriG document §3.

TriG parser §3.

§ F.2 Terms defined by reference

[*BCP47*] defines the following:

section 2.2.9

[*I18N-GLOSSARY*] defines the following:

surrogate code points

Unicode code point

Unicode scalar values

[*LANG-SUBTAG-REGISTRY*] defines the following:

registered language subtags

registered language tags

[*RDF12-CONCEPTS*] defines the following:

- asserted
- blank node
- datatype IRI
- Default Graph
- directional language-tagged strings
- identifier
- ill-typed
- initial text direction
- IRI
- language tag
- lexical form
- literals
- named graphs
- object type
- RDF dataset
- RDF graph
- RDF string
- RDF term
- reifier
- Relative IRI references
- triple
- triple terms

[*RDF12-TURTLE*] defines the following:

- annotation syntax
- Object Lists
- Predicate Lists
- reifying triples
- Simple Triples
- Turtle language

[*RFC3986*] defines the following:

- 5.1.3, Base "URI from the Retrieval URI"
- section 3.3
- section 5.1.1, "Base URI Embedded in Content"
- Section 5.1.2, "Base URI from the Encapsulating Entity"
- section 5.1.4, "Default Base URI"

[*RFC3987*] defines the following:

[RFC3987] defines the following:

syntactic restrictions

§ G. References

§ G.1 Normative references

[BCP47]

Tags for Identifying Languages. A. Phillips, Ed.; M. Davis, Ed. IETF. September 2009. Best Current Practice. URL: https://www.rfc-editor.org/rfc/rfc5646

[EBNF-NOTATION]

EBNF Notation. Tim Bray; Jean Paoli; Michael Sperberg-McQueen; Eve Maler; François Yergeau et al. W3C. W3C Recommendation. URL: https://www.w3.org/TR/xml/#sec-notation

[I18N-GLOSSARY]

Internationalization Glossary. Richard Ishida; Addison Phillips. W3C. 17 October 2024. W3C Working Group Note. URL: https://www.w3.org/TR/i18n-glossary/

[RDF12-CONCEPTS]

RDF 1.2 Concepts and Abstract Data Model. Gregg Kellogg; Olaf Hartig; Pierre-Antoine Champin; Andy Seaborne. W3C. 4 November 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-concepts/

[RFC2119]

Key words for use in RFCs to Indicate Requirement Levels. S. Bradner. IETF. March 1997. Best Current Practice. URL: https://www.rfc-editor.org/rfc/rfc2119

[RFC3629]

UTF-8, a transformation format of ISO 10646. F. Yergeau. IETF. November 2003. Internet Standard. URL: https://www.rfc-editor.org/rfc/rfc3629

[RFC3986]

Uniform Resource Identifier (URI): Generic Syntax. T. Berners-Lee; R. Fielding; L. Masinter. IETF. January 2005. Internet Standard. URL: https://www.rfc-editor.org/rfc/rfc3986

[RFC3987]

Internationalized Resource Identifiers (IRIs). M. Duerst; M. Suignard. IETF. January 2005. Proposed Standard. URL: https://www.rfc-editor.org/rfc/rfc3987

[RFC8174]

Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words. B. Leiba. IETF. May 2017. Best Current Practice. URL: https://www.rfc-editor.org/rfc/rfc8174

[UNICODE]

The Unicode Standard. Unicode Consortium. URL: https://www.unicode.org/versions/latest/

§ G.2 Informative references

[LANG-SUBTAG-REGISTRY]

Language Subtag Registry. IANA. URL: http://www.iana.org/assignments/language-subtag-registry/language-subtag-registry

[RDF12-INTEROP]

RDF 1.2 Interoperability. Pierre-Antoine Champin. W3C. W3C Editor's Draft. URL: https://w3c.github.io/rdf-interop/spec/

[RDF12-N-QUADS]

RDF 1.2 N-Quads. Gregg Kellogg; Dominik Tomaszuk. W3C. 7 November 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-n-quads/

[RDF12-N-TRIPLES]

RDF 1.2 N-Triples. Gregg Kellogg; Dominik Tomaszuk. W3C. 7 November 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-n-triples/

[RDF12-NEW]

What's New in RDF 1.2. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/rdf-new/spec/

[RDF12-PRIMER]

RDF 1.2 Primer. Niklas Lindström; Pierre-Antoine Champin. W3C. 3 April 2025. DNOTE. URL: https://www.w3.org/TR/rdf12-primer/

[RDF12-SCHEMA]

RDF 1.2 Schema. Dominik Tomaszuk; Timothée Haudebourg. W3C. 3 October 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-schema/

[RDF12-SEMANTICS]

RDF 1.2 Semantics. Peter Patel-Schneider; Dörthe Arndt; Enrico Franconi. W3C. 11 November 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-semantics/

[RDF12-TURTLE]

RDF 1.2 Turtle. Gregg Kellogg; Dominik Tomaszuk. W3C. 7 November 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-turtle/

[RDF12-XML]

RDF 1.2 XML Syntax. Gregg Kellogg. W3C. 7 November 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-xml/

[RFC3023]

XML Media Types. M. Murata; S. St. Laurent; D. Kohn. IETF. January 2001. Proposed Standard. URL: https://www.rfc-editor.org/rfc/rfc3023

[SPARQL12-CONCEPTS]

SPARQL 1.2 Concepts. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/sparql-concepts/spec/

[SPARQL12-ENTAILMENT]

SPARQL 1.2 Entailment Regimes. Peter Patel-Schneider. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-entailment/

[SPARQL12-FEDERATED-QUERY]

SPARQL 1.2 Federated Query. Ruben Taelman; Gregory Williams. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-federated-query/

[SPARQL12-GRAPH-STORE-PROTOCOL]

SPARQL 1.2 Graph Store Protocol. Andy Seaborne; Thomas Pellissier Tanon. W3C. 19 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-graph-store-protocol/

[SPARQL12-NEW]

What's New in SPARQL 1.2. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/sparql-new/spec/

[SPARQL12-PROTOCOL]

SPARQL 1.2 Protocol. Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-protocol/

[SPARQL12-QUERY]

SPARQL 1.2 Query Language. Olaf Hartig; Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 12 November 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-query/

[SPARQL12-RESULTS-CSV-TSV]

SPARQL 1.2 Query Results CSV and TSV Formats. Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-csv-tsv/

[SPARQL12-RESULTS-JSON]

SPARQL 1.2 Query Results JSON Format. Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-json/

[SPARQL12-RESULTS-XML]

SPARQL 1.2 Query Results XML Format. Ruben Taelman; Dominik Tomaszuk; Thomas Pellissier Tanon. W3C. 27 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-xml/

[SPARQL12-SERVICE-DESCRIPTION]

SPARQL 1.2 Service Description. Ruben Taelman; Gregory Williams. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-service-description/

[SPARQL12-UPDATE]

SPARQL 1.2 Update. Ruben Taelman; Andy Seaborne; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-update/

[UNICODE-SECURITY]

Unicode Security Considerations. Mark Davis; Michel Suignard. Unicode Consortium. 19 September 2014. Unicode Technical Report #36. URL: https://www.unicode.org/reports/tr36/tr36-15.html

↥